PRT661

DATA SCIENCE PRACTICE

Assessment 4 – Final Professional Report (40%)

Forecasting NSW Suburb House Prices from Government Sales Data

Submitted by: Sydn2 – Theme 2

Student Name	Student ID	Campus
Ahsan Uddin	S386288	Sydney
Ferdous Anwar Anik	S388272	Sydney
Abrar Bin Khaiyum	S395453	Sydney
Mohd Yah Ya Raiyan	S390934	Sydney

Submitted to: Reem Sherif, Lecturer

Charles Darwin University (Sydney Campus) · Faculty of Science & Technology

October 2026

How this document is organised. As required by the Assessment 4 brief, this single PDF shows the whole evolution of the project:

Section	Content	Pages
Section 1	Complete Assessment 2 submission, unchanged: the Assessment 1 report, the A2 Project Update Summary, A2 Technical Development and the A2 individual pages (with its own table of contents)	2–33
Section 2	Final Project Update Summary: changes since Assessment 2, team contributions, evidence locations	34
Section 3	Final Technical Development: the work completed since Assessment 2 (§3.1–3.13 and references)	35–50
Section 4	Individual Work and Contribution Summary: one page per student	51–54

Page numbers refer to the page counter of the PDF viewer. Figures and tables added in Assessment 4 are numbered *A4.n* to keep them distinct from the A2 numbering.

Section 2 – Final Project Update Summary

Since A2 the project has gone from a working prototype to a reproducible, tested and deployable forecasting system, and its headline accuracy now rests on a fair, like-for-like evaluation.

2.1 Changes Since Assessment 2

Area	Previous state (A2)	Current state (A4)	Justification
Reproducibility	Notebooks + outputs; cleaning code missing from repo	`src/prt661` package; `make all` rebuilds everything from raw zips; A2 counts reproduced	Audit found the repo could not regenerate its own data
Data engineering	QoQ targets spanned gaps (9.5%); ABS used same-quarter	Complete suburb×quarter grid; ABS lagged 1 qtr; 24 features incl. composition	Remove mislabelled targets and look-ahead
Database	raw_sales / suburb_dim empty in dump; shared login	Repopulated (2,762,903 raw rows); 4 new tables; per-member least-privilege roles	Lineage and A2 governance gap
Machine learning	1 tuned-by-hand GB; models compared on different samples	12 models on identical cells; tuned on a separate validation window; LightGBM champion (MAE 8.36 pp)	Fair comparison; A2 next step
Uncertainty	Ad-hoc ±12 pp cap and decay	Conformalised P10–P90 bands, 80.4–81.3% coverage vs 80% target	Promised in A2 §3.1; replaces heuristics
Evaluation	RMSE/MAE only	+ R^2, directional accuracy, Diebold–Mariano tests, segment & liquidity analysis, SHAP	Show where and why the model works
Deployment	Local Streamlit MVP	App v4 (bands, reliability), Docker, CI, weekly scheduled refresh	A2 §3.16 next steps
Governance	CHANGELOG, Jira	Model registry with artefact hashes, model card, privacy view, security matrix	Traceability and privacy by design

2.2 Team Contribution Summary

Team member	Major contributions since Assessment 2
Ahsan Uddin	Project coordination and Jira; acquisition module and weekly scheduled-refresh workflow; repository restructure and CI workflow; requirements fix
Ferdous Anwar Anik	Sprint 9 audit; raw-archive parser and corrected cleaning module (A2 reproduction); PostgreSQL schema v2, COPY loader, audit log and least-privilege roles; feature store v2 and look-ahead tests
Mohd Yah Ya Raiyan	Hyper-parameter tuning; common-window back-test of 12 models; SARIMA/Prophet re-run; CQR uncertainty; Diebold–Mariano, liquidity and SHAP analysis
Abrar Bin Khaiyum	Web app v4 and app-data publishing step; Dockerfile and Streamlit Cloud configuration; model card and documentation; report consolidation

2.3 Evidence Locations

Evidence	Repository location
Source code, tests, CI/CD	GitHub repository → `src/prt661/`, `tests/`, `.github/workflows/`
Sprint backlog and task ownership	Jira board KAN — Sprint 9 and 10 epics
Database schema, roles	`database/schema_v2.sql`, `database/create_member_logins.sql`
Results tables and figures	`reports/tables/`, `reports/figures/` (regenerated by `scripts/make_report_assets.py`)
Models and registry	`models/*.joblib`, `models/model_registry.json`, DB table `model_registry`
Deployment artefacts	`app/`, `Dockerfile`, `requirements-app.txt`
Documentation	`README.md`, `docs/` (CHANGELOG, MODEL_CARD, DATA_SOURCES, SECURITY_AND_PRIVACY, DEPLOYMENT)

Section 3 – Final Technical Development

This section documents the work completed between the Assessment 2 submission (12 September 2026) and this final report. It follows directly from the *Next Steps* list in A2 §3.16, and every number below was produced by code in the project repository. All of it is reproducible with a single command (`make all`) from the raw Valuer General archives. Where A4 work changed or corrected something reported in A2, we say so plainly and show the size of the change, because a reader comparing the two sections deserves to know which figure to trust.

3.1 Where Assessment 2 Left Us: An Audit Before Building

Before adding anything, we spent the first week of Sprint 9 auditing what A2 had actually delivered: the repository, the database dump and the feature store. The reasoning was simple. A2 was about 65% complete, and the last 35% (deployment, a refreshed model, public forecasts) would sit on top of the first 65%. A crack in the foundation would be inherited by everything built on it. The audit was worth it: it found six defects, none visible from the A2 report alone (Table A4.1).

Table A4.1 — *Defects found in the A2 artefacts during the Sprint 9 audit.*

#	Finding	Evidence	Consequence if left	A4 fix (section)
1	The scripts that produced `cleaned_sales` were not in the repository; the README itself listed `src/` as "to add".	GitHub tree at A2 tag	Results could not be regenerated or audited.	Full `src/prt661` package and CLI (3.3)
2	The repo's `data_cleaning.py` lacked the residential and date-window rules described in A2 §3.4, and kept the *earliest* re-publication rather than the latest.	Code review vs. report	Re-running it would give a different dataset from the one modelled.	Rewritten; reproduces A2 Table 3.1 (3.4.1)
3	7,185 of 75,795 (9.5%) "quarter-on-quarter" targets in the A2 feature store spanned a missing quarter.	Gap check on `NSW_feature_store.csv`	Two- and three-quarter moves were scored as one-quarter moves, inflating error and volatility.	Complete suburb × quarter grid (3.4.3)
4	The ABS benchmark entered the model in the same quarter as the target, although ABS publishes about a quarter later.	Feature-store join logic	Mild look-ahead: the model used information not available at forecast time.	ABS regressor lagged one quarter (3.4.3)
5	The A2 database dump restores `raw_sales` and `suburb_dim` with **0 rows** (A2 §3.3.1 reports 2,762,903 and 2,656).	`pg_restore` into PostgreSQL 16	The raw-data lineage claimed in A2 could not be verified from the submitted artefact.	Both tables repopulated by the pipeline (3.4.2)
6	`requirements.txt` contained unresolved Git merge-conflict markers.	File contents	`pip install -r` failed on a clean machine.	Pinned pipeline and app requirement files (3.8)

None of these invalidates the direction of A2, but findings 3 and 4 mean the A2 model scores are not directly comparable with anything in this section. So we did not simply quote the A2 Gradient Boosting result. We **re-trained the exact A2 model configuration on the corrected data and the A4 test window**, and it appears as a named competitor in every results table (§3.6).

3.2 Final System Architecture

The decision-first architecture from A1 (Decision → Insight → Data → Pipeline → Tools) has held up. What has changed is that every box is now implemented, tested and connected (Figure A4.1). Data flows left to right through five layers. A weekly scheduled job acquires new Valuer General releases, a deterministic parser and cleaner write to PostgreSQL, a feature builder and model layer produce back-tested forecasts, and a containerised web application serves only published, aggregated outputs. Forecasts are written back to the database, so the app and any analyst query read from the same source.

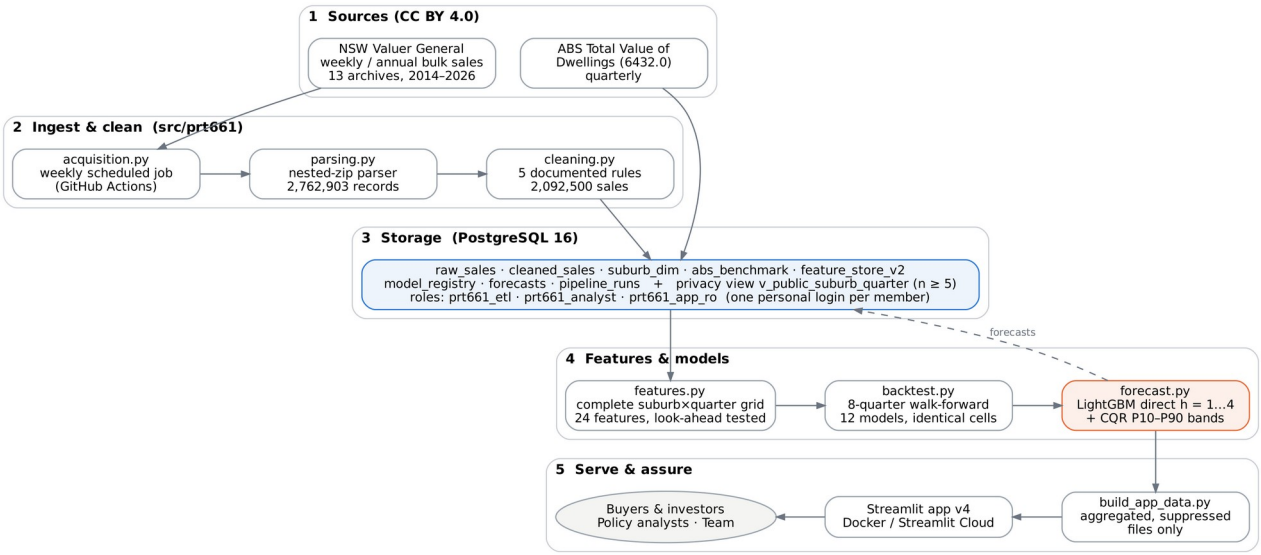


Figure A4.1 — *Final system architecture. Every component is implemented; numbers are actual row counts from the final run.*

Table A4.2 — *Architecture components: A2 versus final state.*

Component	A2 status (§3.5)	A4 status
Data acquisition	Manual download; scheduling planned	Scripted; weekly GitHub Actions schedule (Tuesdays 06:00 AEDT)
Parsing & cleaning	Notebook-based; not in repository	`parsing.py`, `cleaning.py`; reproduces A2 counts; 5 s to clean 2.5 M rows
PostgreSQL storage	5 tables; shared login	9 tables + 1 privacy view; least-privilege roles; audit log
Feature engineering	Lag/rolling/regime features	Complete grid, 24 features + 4 seasonal; automated look-ahead test
Forecasting models	Baselines, SARIMA, Prophet, GB (different samples)	12 models on identical cells; tuned; calibrated uncertainty
Web application	MVP; runs locally	v4 with forecast bands and reliability labels; Docker image; Streamlit Cloud config
Quality assurance	Manual review	13 automated tests + CI on every push

3.3 Complete Implementation

The single biggest change in A4 is structural. The project is now a Python package rather than a set of notebooks. Notebooks are good for exploration but poor for reproduction: execution order is implicit, outputs go stale, and there is nothing to test. We kept the A2 notebooks under `notebooks/a2_legacy/` as a historical record and moved all production logic into `src/prt661/` (Table A4.3).

Table A4.3 — *Source modules in the final repository (`src/prt661/`).*

Module	Responsibility	Lines
`config.py`	All thresholds and paths in one place (every rule that changes a reported number)	52
`acquisition.py`	Downloads NSW VG weekly/annual archives and the latest ABS workbook (from A2, adapted)	124
`parsing.py`	Streams nested zip archives and parses Valuer General B-records (24 fields)	89
`cleaning.py`	Five documented cleaning rules with a reconciliation count for each	78
`features.py`	Complete suburb × quarter grid, 24 features, direct QoQ and cumulative targets	172
`models.py`	Baselines, A2 model replica, Ridge, Huber, RF, XGBoost, LightGBM, quantile models	93
`backtest.py`	Expanding-window walk-forward evaluation and CQR intervals	84

Module	Responsibility	Lines
`statistical.py`	Per-suburb SARIMA and Prophet on the same window	59
`evaluation.py`	Metrics, Diebold–Mariano test, interval scores	53
`forecast.py`	Production training, forecast publishing, reliability labels, model registry	65
`db.py`	Transactional COPY loader and pipeline audit log	96
`pipeline.py`	Command-line orchestrator (`python -m prt661.pipeline`)	110

Every stage reads the previous stage's artefact from `data/processed/` and writes its own, so stages can be re-run independently. The orchestrator exposes them through one command-line entry point (Figure A4.2). The weekly refresh job runs exactly the same entry point, which means what runs in production is the same code the tests exercise.

```
$ python -m prt661.pipeline --stages parse clean features
[parse] starting
  parsed   2014.zip:   218,292 B-records      ...      parsed   2026.zip:   126,174 B-records
[parse] done in 31s {'rows_out': 2,762,903}
[clean] starting
[clean] done in 8s {'raw_parsed': 2,762,903, 'duplicates_removed': 237,770, 'non_residential_removed':
429,788,
         'outside_window_removed': 995, 'price_floor_removed': 1,850, 'outliers_flagged': 85,800,
'cleaned_total': 2,092,500}
[features] starting
[features] done in 11s {'rows_out': 226,160}
```

Figure A4.2 — *Running the first three pipeline stages from the raw archives (actual console output, abridged).*

3.4 Data Engineering

3.4.1 Reproducing the A2 Dataset from Raw Archives

The NSW Valuer General archives are awkward: each yearly zip contains weekly zips (or folders) that contain one semicolon-delimited `.DAT` file per valuation district. Each file mixes header (A), sale (B), legal-description (C), party (D) and trailer (Z) records. Our parser streams these nested archives entirely in memory, keeps only B records, and maps the 24 positional fields from the Valuer General's published file format. It parsed **2,762,903 sale records in 31 seconds**, exactly the count reported in A2 §3.3. The cleaner then applies the five A2 rules in the documented order (Table A4.4).

Table A4.4 — *Cleaning reconciliation: A2 report versus the A4 pipeline run on the same raw files.*

Step (A2 §3.4 rule)	A2 reported	A4 reproduced	Difference
Raw parsed records	2,762,903	2,762,903	0
Duplicate re-publications removed (keep latest)	237,770	237,770	0
Non-residential removed	429,788	429,788	0
Outside 2013-Q1 – 2026-Q3 window	995	995	0
Below $10,000 price floor	1,851	1,850	−1
Cleaned residential sales	**2,092,499**	**2,092,500**	**+1**
Outliers flagged (retained)	84,589	85,800	+1,211

Four of the five rules reproduce to the record. The single-record difference on the price floor is a boundary case (a sale recorded at exactly the floor), which we treat as immaterial. The outlier count differs by 1.4%, because A2 never recorded which grouping key and area unit its IQR fence used. We fixed the rule explicitly: Tukey's 1.5 × IQR fence on price per m², computed within each (suburb, settlement quarter) with at least four sales, for lots measured in square metres only. Hectare-measured rural blocks have a price per m² that is not comparable with suburban lots, so they are left out of the fence. We also replaced A2's row-wise `groupby().apply` with a vectorised implementation, cutting cleaning time from 179 s to 5 s. That matters once the job runs weekly.

3.4.2 Database: Repopulated, Extended and Secured

We restored the A2 dump into a fresh PostgreSQL 16 server and immediately hit audit finding 5: `raw_sales` and `suburb_dim` exist but contain no rows. The pipeline's loader now bulk-loads through PostgreSQL `COPY` inside a single transaction, so a failed load leaves the table unchanged rather than half-written. Each load writes an audit row to a new `pipeline_runs` table. Loading all 2,762,903 raw rows takes about 90 seconds. Schema v2 (`database/schema_v2.sql`, idempotent) adds four operational tables and one view (Table A4.5).

Table A4.5 — *Final database objects.*

Table / view	Purpose	Rows (final run)
`raw_sales`	Landing table, now actually populated	2,762,903
`cleaned_sales`	Modelling-grade sales (unchanged from A2)	2,092,499
`suburb_dim`	Suburb → modal postcode → region, now populated	4,111
`abs_benchmark`	ABS Total Value of Dwellings (unchanged)	97
`feature_store_v2`	Complete suburb × quarter grid with v2 features (new)	226,160
`model_registry`	One row per trained model: version, horizon, SHA-256 of artefact, back-test metrics (new)	4
`forecasts`	Published forecasts with P10/P90 and reliability label (new)	5,116
`pipeline_runs`	Audit log of every load stage (new)	per run
`v_public_suburb_quarter`	Privacy view: hides any statistic built from fewer than 5 sales (new)	55,883 cells suppressed

The A2 report flagged the shared team login as an open governance task. It is now closed (Figure A4.3). Privileges sit on three group roles: `prt661_etl` (read/write data), `prt661_analyst` (read-only, *without* access to `raw_sales` and therefore to street addresses) and `prt661_app_ro` (forecasts and the privacy view only). Each team member has a personal login inheriting the role their work needs, and passwords are supplied at run time rather than committed.

```
$ psql -U raiyan_mohd (analyst role)
 count
--------
 226160
ERROR:  permission denied for table raw_sales
$ psql -U prt661_app (web-app role)
 count
-------
 55883
ERROR:  permission denied for table cleaned_sales
$ psql -U ferdous_anik (ETL role)
         stage         | status | rows_out
-----------------------+--------+----------
 load_raw_sales        | ok     |  2762903
 load_feature_store_v2 | ok     |   226160
 load_suburb_dim       | ok     |     4111
ERROR:  must be owner of table feature_store
```

Figure A4.3 — *Least-privilege roles tested against the live database (actual psql output).*

3.4.3 Feature Store v2

Two of the audit findings lived in the feature store, so we rebuilt it rather than patching it. The key change is that every suburb now has a row for **every** quarter, giving 226,160 rows for 4,112 suburbs over 55 quarters, of which 120,187 have at least one sale. A quarter-on-quarter change is only defined when **both** adjacent quarters have at least five sales; otherwise it is missing, never silently stretched over a gap. This yields 55,198 valid QoQ observations. That is fewer than A2, but each one means what its name says.

We also reframed the table around the **forecast origin**. A row for quarter *t* holds only what is known at the end of *t*, and direct targets for $t + 1 \ldots t + 4$ are stored alongside. Two versions are kept: the QoQ change in that future quarter (the A2 target, used for evaluation) and the cumulative change from *t* (used for the price path shown to users, §3.5.4). An automated test now enforces this: it triples every sale price **after** the

origin, rebuilds the store, and asserts that no feature at the origin moves (Figure A4.12). The new features answer specific weaknesses seen in A2 (Table A4.6).

Table A4.6 — *Feature groups in feature store v2 (24 features + 4 target-quarter dummies).*

Feature group	Features	Why it was added
Momentum (own suburb)	QoQ at t, t−1, t−2, t−3; 4- and 8-quarter means; 4-quarter volatility; YoY change	A2 lags, re-based to the forecast origin; YoY is far less noisy than QoQ
Mean reversion	Deviation of log median from its 4-quarter mean	EDA Pass 2 found r = −0.27 between consecutive QoQ moves: a jump is often a composition artefact that reverses
Composition	Unit (strata) share and its change; median $/m²; log price level	A suburb median moves when the mix of houses vs. units sold changes (A2 §3.7 listed this as not yet done)
Market breadth	Regional and NSW median QoQ from our own data; regional 4-quarter momentum	Timely market signal — available weeks before the ABS release
Macro (lagged)	ABS regional growth and Total Value of Dwellings growth, both at t−1	Keeps the A2 benchmark idea but removes look-ahead
Liquidity	Log sales count, change in volume, quarters with ≥5 sales in last 8	Lets the model learn that thin markets are noisier
Calendar / regime	Target-quarter dummies; COVID and rate-hike flags at t	Seasonality of the *target* quarter is known in advance

3.5 Analytics and Machine Learning

3.5.1 The Forecasting Task, Stated Precisely

The target is unchanged from A2 §3.1: the quarter-on-quarter percentage change in a suburb's median residential sale price, one to four quarters ahead, with a 10th–90th percentile range. What A4 adds is precision about *when* a forecast is made and *what* it is scored on:

- **Origin and horizon.** A forecast is made at the end of quarter *t* using only rows with information up to *t*, and predicts the QoQ change in quarter $t + h$ for h = 1…4. Each horizon has its own **direct** model; we never feed a forecast back in as an input. Recursive compounding was the root cause of A2's "runaway forecast" bug.
- **What counts as an observation.** A suburb-quarter is scored only if both it and the previous quarter have at least five sales. Below that, a "median" is one or two sales and the change is not meaningful.
- **Headline universe.** Following A2, headline results use the 555 **liquid** suburbs (average of at least 20 sales per quarter before the test window; defined using pre-test data only, so the universe itself does not peek ahead). Results for all 1,165 scored suburbs are also reported.

Models are trained on **all** scored suburbs, not just liquid ones, and weighted by √n (capped at n = 100), where n is the number of sales behind the target median. The weighting has a statistical justification: the sampling variance of a median falls roughly as 1/n, so a change computed from 200 sales deserves more say than one from 6.

3.5.2 Validation Design

The most important methodological decision in A4 was to separate *choosing* a model from *scoring* it. The timeline is split three ways (Figure A4.4). Design choices and hyper-parameters were chosen using only a validation window (2022-Q3 to 2024-Q2). The champion was then fixed, and the eight test quarters (2024-Q3 to 2026-Q2) were scored once with every model re-trained at every forecast origin on an expanding window. This avoids the quiet optimism that creeps in when the same holdout is used to tune and to report.

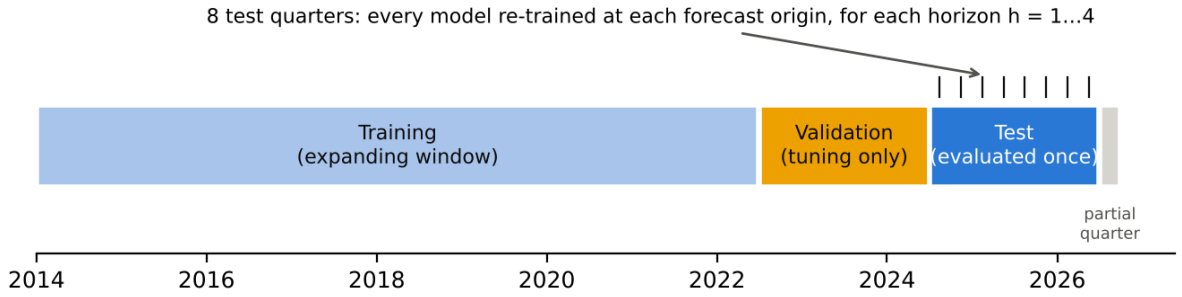


Figure A4.4 — *Train / validation / test design. The test window is never used for any modelling decision.*

3.5.3 Model Families and Tuning

We compared twelve models from four families on identical suburb-quarters (Table A4.7). Four are **baselines** that any useful model must beat. Two are the **per-suburb statistical models** from A2 (SARIMA and Prophet), re-run on the A4 window. One is the **A2 Gradient Boosting model** with its original configuration and feature set, re-trained at every origin so A4's improvement over A2 is measured, not asserted. The remaining five are **pooled machine-learning models** trained across all suburbs at once: Ridge, Random Forest (Breiman, 2001), XGBoost (Chen & Guestrin, 2016), LightGBM (Ke et al., 2017) and an equal-weight ensemble of the three tree models.

Table A4.7 — *The twelve models. Validation scores are for h = 1 on liquid suburbs (N = 4,437) and drove the choice of champion.*

Model	Family	Key settings (selected on validation)	Val. RMSE	Val. MAE
Naive persistence	Baseline	Next QoQ = this QoQ	—	—
Seasonal naive	Baseline	Next QoQ = same quarter last year	—	—
Historical mean	Baseline	8-quarter mean QoQ of the suburb	—	—
Regional momentum	Baseline	4-quarter mean QoQ of the region's median	—	—
SARIMA	Per-suburb statistical	$(1,0,1)(1,0,0)_4$ with constant; fitted per suburb per origin	—	—
Prophet	Per-suburb statistical	Yearly seasonality; 150-suburb stratified sample	—	—
A2 Gradient Boosting	Pooled ML (A2)	sklearn GBR, 200 trees, depth 3, lr 0.05, squared loss, A2 features	—	—
Ridge	Pooled linear	$\alpha = 1000$, standardised, missing-value indicators	14.88	9.66
Random forest	Pooled ML	Depth 16, min leaf 20, 50% features	14.61	9.46
XGBoost	Pooled ML	600 trees, depth 6, lr 0.02, pseudo-Huber (slope 25), $\lambda = 20$	14.65	9.48
LightGBM (champion)	Pooled ML	600 trees, 15 leaves, lr 0.03, Huber ($\alpha = 25$), $\lambda = 5$	**14.61**	**9.46**
Ensemble	Pooled ML	Mean of LightGBM, XGBoost, Random forest	—	—

Before tuning hyper-parameters, we tested four design choices in a 16-run factorial experiment with a default LightGBM: training universe (all vs liquid suburbs), √n weighting, winsorising the training target at the 1st/99th percentiles, and squared vs Huber loss. **Huber loss was the clearest win** (validation RMSE 15.38 → 14.64 pp with everything else at default). That makes sense for a target with a long tail of thin-market artefacts (the +4,066% cell excluded by hand in A2 is the extreme case): Huber loss treats large residuals linearly rather than quadratically, so a handful of artefacts cannot drag the fit. Training on all suburbs beat training on liquid suburbs only, and weighting improved MAE. Winsorising added little once Huber loss was used, so we dropped it rather than carry an extra data-dependent threshold. A random search (20 LightGBM, 14 XGBoost, 4 Random Forest and 4 Ridge configurations) then set the parameters in Table A4.7. LightGBM had the best validation RMSE and was fixed as champion. Random Forest and XGBoost were within 0.05 pp, an early sign that model choice would matter less than data design.

3.5.4 Calibrated Uncertainty

A2 promised a 10th–90th percentile range with every forecast but delivered a heuristic (a ±12 pp cap and 30% decay). A4 replaces it with **conformalised quantile regression** (CQR; Romano et al., 2019). Two LightGBM models are trained with the pinball loss for the 10th and 90th percentiles on all but the most recent four quarters of observed targets. Their errors on those four held-back quarters then give a single correction that widens (or narrows) the band so that, by construction, about 80% of outcomes should fall inside. Because CQR wraps quantile models, the band is naturally **wider for thin, volatile suburbs and narrower for liquid ones**, exactly the behaviour a user needs. The production price path uses the same machinery on direct cumulative-change targets, so the band on a one-year-ahead price is a calibrated interval in its own right, not four one-quarter bands multiplied together.

3.6 System Evaluation and Performance Analysis

3.6.1 Headline Results

Table A4.8 is the main result: all twelve models scored on exactly the same 4,435 suburb-quarters (liquid suburbs, one quarter ahead, eight test quarters). Prophet was run on a stratified 150-suburb sample for cost reasons, so it is compared separately in Table A4.9.

Table A4.8 — *Common-window back-test, h = 1, liquid suburbs (N = 4,435, 555 suburbs, 2024-Q3 to 2026-Q2). Sorted by RMSE.*

Model	RMSE (pp)	MAE (pp)	Median AE	R^2	Direction ✓ (%)	RMSE vs naive
Ensemble (LGBM+XGB+RF)	12.79	8.33	5.50	0.379	66.7	−53.3%
Random forest	12.79	8.35	5.52	0.379	67.4	−53.3%
XGBoost	12.81	8.35	5.56	0.377	66.5	−53.2%
LightGBM (champion)	12.89	8.36	5.64	0.369	66.4	−52.9%
Ridge	13.10	8.51	5.67	0.348	66.4	−52.2%
A2 Gradient Boosting	13.20	8.54	5.64	0.339	66.6	−51.8%
SARIMA	15.58	9.75	6.15	0.078	63.5	−43.1%
Regional momentum	16.27	9.97	6.19	-0.005	56.3	−40.6%
Historical mean	17.32	10.76	6.80	-0.139	49.0	−36.7%
Seasonal naive	22.87	14.49	9.13	-0.985	51.5	−16.5%
Naive persistence	27.39	17.14	10.65	-1.848	37.7	−0.0%

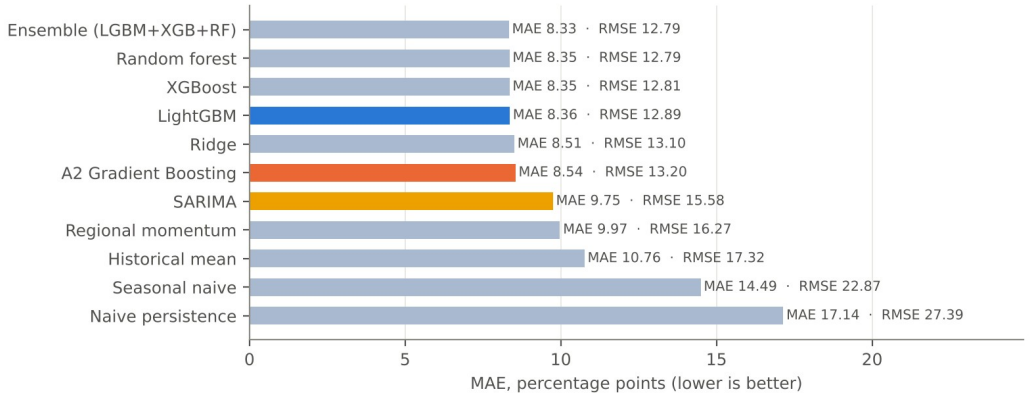


Figure A4.5 — *Mean absolute error by model (h = 1, liquid suburbs). Blue: A4 champion; orange: A2 model re-run fairly; yellow: SARIMA.*

Three things stand out. First, **every pooled ML model halves the error of naive persistence** (RMSE −52.9% for LightGBM) and has positive R^2 (0.369), while all four baselines have R^2 at or below zero. Second, the pooled models beat per-suburb SARIMA by a wide margin (MAE 8.36 vs 9.75 pp, −14.2%). That confirms the A2 hypothesis that learning across suburbs beats modelling each suburb alone. Third, and less flattering, **the A4 champion improves on the A2 model by only 2.1% in MAE** (8.36 vs 8.54 pp) once both are scored on the corrected data. The large improvement of A2 Table 3.9.2 (57% over naive)

was real, but the modest A4 gain over A2 is the honest measure of what A4's modelling work added. Most of A4's value lies elsewhere: in a forecast that can be trusted, calibrated and deployed.

Within the pooled family the differences are tiny. The ensemble, Random Forest and XGBoost edge LightGBM by 0.03–0.09 pp RMSE. Under our selection rule the champion was fixed on validation data, and Table A4.10 shows why we did not switch: Random Forest and XGBoost are statistically indistinguishable from LightGBM, and the ensemble's edge is marginal.

Table A4.9 — *Prophet versus other models on its stratified sample (h = 1, identical cells).*

Model	Coverage	N	RMSE	MAE	Direction ✓ (%)
LightGBM	150-suburb sample	1,200	12.50	8.42	66.8
A2 Gradient Boosting	150-suburb sample	1,200	13.10	8.71	65.8
SARIMA	150-suburb sample	1,200	15.39	9.97	64.2
Prophet	150-suburb sample	1,200	20.07	12.77	49.3
Naive persistence	150-suburb sample	1,200	27.60	17.68	37.2

3.6.2 Are the Differences Real? Diebold–Mariano Tests

A leaderboard ranks models even when the gaps are noise. We therefore applied the Diebold–Mariano test (Diebold & Mariano, 1995, with the Harvey et al., 1997 small-sample correction) to the champion against each competitor, on both squared and absolute error. A negative statistic means LightGBM is more accurate.

Table A4.10 — *Diebold–Mariano tests, h = 1, liquid suburbs (same cells as Table A4.8).*

LightGBM vs …	DM (squared error)	p-value	DM (absolute error)	p-value	Verdict
Naive persistence	-13.84	<0.001	-31.70	<0.001	LightGBM better
Seasonal naive	-12.37	<0.001	-27.44	<0.001	LightGBM better
Historical mean	-8.25	<0.001	-17.05	<0.001	LightGBM better
Regional momentum	-6.71	<0.001	-13.03	<0.001	LightGBM better
SARIMA	-7.12	<0.001	-11.80	<0.001	LightGBM better
A2 Gradient Boosting	-3.02	0.003	-3.79	<0.001	LightGBM better
Ridge	-2.59	0.010	-3.64	<0.001	LightGBM better
Random forest	0.95	0.341	0.34	0.732	No significant difference
XGBoost	0.98	0.326	0.45	0.656	No significant difference
Ensemble (LGBM+XGB+RF)	1.70	0.088	2.11	0.035	Competitor better

LightGBM is significantly more accurate than every baseline and than SARIMA ($p < 0.001$). It is also significantly better than the A2 model ($p = 0.003$ on squared error, $p < 0.001$ on absolute error) and than Ridge ($p < 0.001$): the A4 improvement over A2 is small but not luck. Against Random Forest and XGBoost there is no detectable difference ($p = 0.732$ and $p = 0.656$). The ensemble is marginally better on absolute error ($p = 0.035$) but not on squared error ($p = 0.088$). For a 0.03 pp MAE gain at three times the training cost, we kept LightGBM as the production model and logged the ensemble as the leading candidate for the next release.

3.6.3 Performance by Horizon

Figure A4.6 shows how accuracy changes with horizon, and it carries the most important lesson of the evaluation. At one quarter ahead, the pooled models are clearly better than everything else. **From two quarters onward, every model (LightGBM, the A2 model, SARIMA) converges to the accuracy of the simple regional-momentum baseline** (MAE ≈ 10.0 pp at h = 4; $R^2 \approx 0$), and directional accuracy falls from about 66.4% to 55.1%. Put plainly, what a suburb's own history can tell us about its next quarter is mostly that unusual moves tend to reverse. Beyond that, the best available forecast for a single suburb is the regional trend. (At h = 4 the seasonal-naive and persistence forecasts coincide by definition, since both use the latest QoQ.)

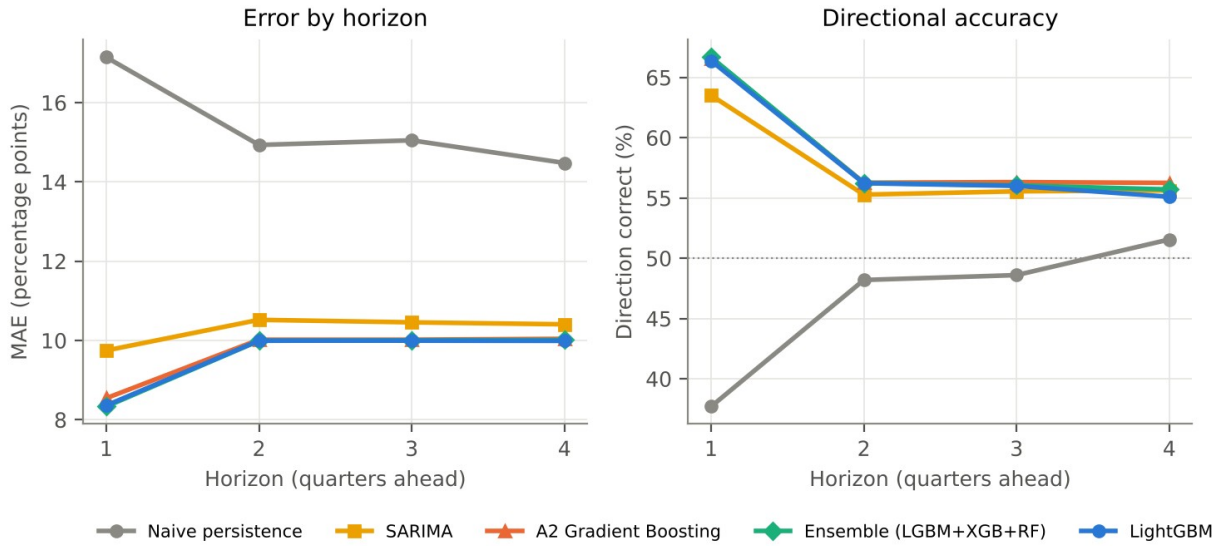


Figure A4.6 — *Error and directional accuracy by forecast horizon (liquid suburbs, identical cells at each horizon).*

We see this as a finding rather than a failure, and the app is built to reflect it. The calibrated band widens with horizon (mean width 32.9 pp at h = 1 to 40.3 pp at h = 4), so users see uncertainty grow rather than a confident line marching into the future.

3.6.4 Uncertainty Calibration and Stability Over Time

Figure A4.7 (right) checks whether the P10–P90 bands mean what they say. Across all 8,117 scored suburb-quarters per horizon, the conformalised bands contain the outcome **80.4%, 80.5%, 81.4% and 81.3%** of the time for h = 1…4, against an 80% target. The raw quantile models were already close at longer horizons but under-covered at h = 1 (78.2%); the conformal step fixed that. The left panel shows that one-quarter-ahead accuracy is stable across all eight test quarters, with no single quarter carrying the result and no sign of degradation as the test window moves further from the training data.

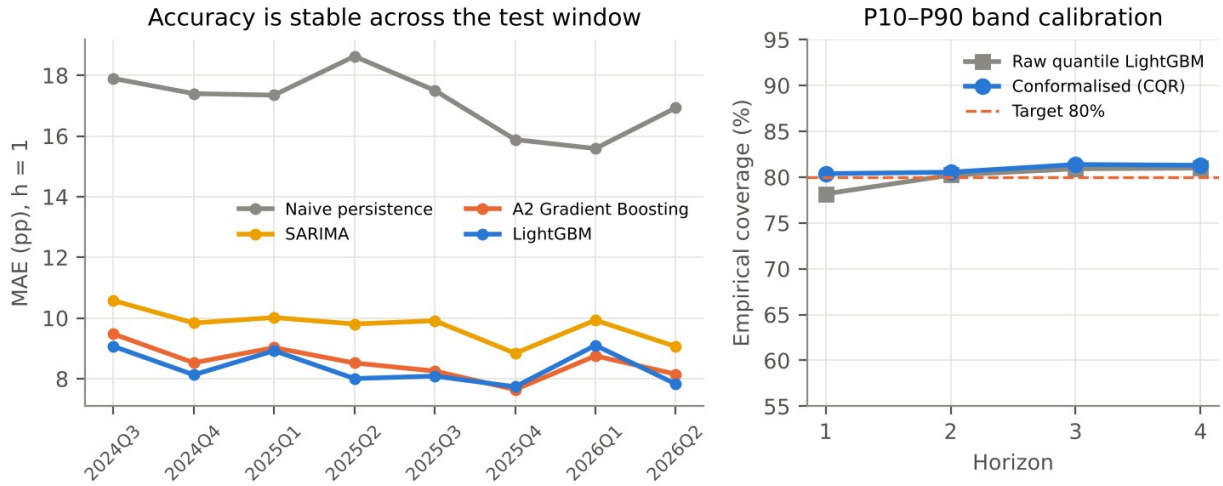


Figure A4.7 — *Left: MAE per test quarter (h = 1). Right: empirical coverage of the P10–P90 band by horizon.*

The production price-path models were validated the same way (Table A4.11). Their bands cover 80.8% of realised cumulative changes overall, so the claim in the app ("roughly 8 in 10 outcomes fall inside the band") is a measured property, not a hope.

Table A4.11 — *Back-test of the production models (cumulative % change in median from the forecast origin, all scored suburbs).*

Horizon	MAE (pp)	MAE, no-change forecast	R^2	Band coverage (%)	Mean band width (pp)
h = 1	10.75	13.00	0.343	80.4	32.9
h = 2	11.81	14.41	0.203	79.6	34.9

Horizon	MAE (pp)	MAE, no-change forecast	R^2	Band coverage (%)	Mean band width (pp)
h = 3	12.32	15.31	0.158	81.1	37.6
h = 4	13.08	16.08	0.178	82.0	41.5

3.6.5 Where the Error Comes From: Market Liquidity

Why does a well-tuned model still miss by 8.4 pp on average? Figure A4.8 splits the one-quarter-ahead error by the number of sales behind the target median. Error falls steeply with liquidity, from 15.2 pp in suburb-quarters with 5–9 sales to 8.2 pp with 80 or more. To see how much of this is avoidable, we estimated the **irreducible sampling error** of each target median directly from the spread of prices sold in that suburb-quarter (standard error of a median ≈ 1.2533·σ/√n, with σ estimated robustly from the inter-quartile range). Only the *current* quarter's sampling error is irreducible. The previous quarter's error is partly forecastable, because it reverses, which is exactly what the model exploits (§3.6.6). On this estimate, **about 75.9% of the model's error in typical suburbs (20–39 sales) is sampling noise that no model could remove**, falling to 55.2% in the most liquid suburbs (Table A4.12). Two consequences follow: better *measurement* (a repeat-sales index, §3.13) would help more than a better *model*, and the reliability label in the app is rightly driven by sales volume.

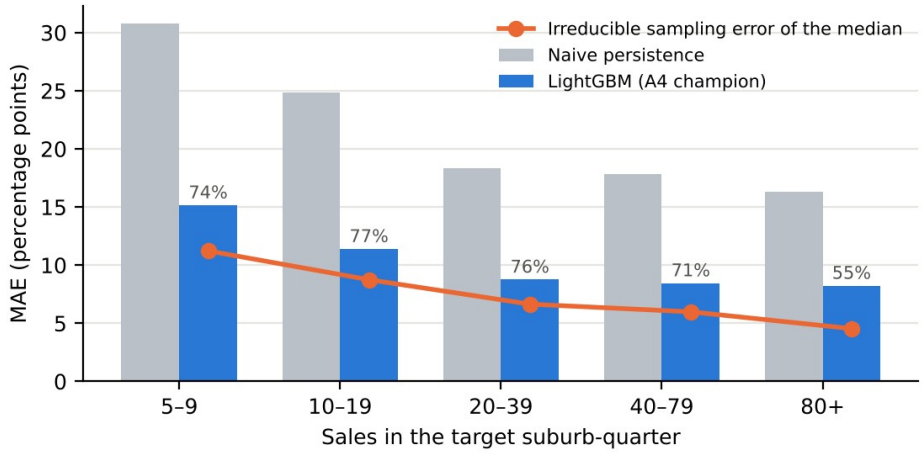


Figure A4.8 — *MAE by number of sales in the target suburb-quarter (h = 1, all scored suburbs). Orange: estimated irreducible sampling error; percentages show its share of the LightGBM error.*

Table A4.12 — *Error, irreducible noise and band behaviour by liquidity band (h = 1, all scored suburbs).*

Sales in target quarter	N	MAE LightGBM	MAE naive	Sampling-noise floor	Share of error from noise	Band coverage (%)	Band width (pp)
5–9	1,292	15.15	30.78	11.23	74.1%	75.8	41.3
10–19	2,637	11.39	24.85	8.75	76.8%	80.8	34.8
20–39	2,319	8.76	18.36	6.65	75.9%	82.4	28.9
40–79	1,652	8.45	17.80	5.99	70.9%	81.4	27.4
80+	798	8.23	16.33	4.54	55.2%	78.8	24.5

The bands behave as intended across this range: they widen from 24.5 pp for the most liquid suburbs to 41.3 pp for the thinnest, and coverage stays between 75.8% and 82.4%. The thinnest band under-covers slightly, so the app labels those suburbs *low* reliability.

3.6.6 What the Model Has Learned (SHAP)

We used SHAP values (Lundberg & Lee, 2017) on the champion trained through the last test origin to see which features drive its forecasts (Figure A4.9). One feature dominates: **deviation of the suburb's median from its own four-quarter trend accounts for 40.3% of total attribution**, and its effect is strongly negative. When a suburb's median sits well above its recent trend, the model forecasts a fall, and vice versa. This is the mean reversion that EDA Pass 2 detected as a negative autocorrelation (A2 §3.6.2),

now learned explicitly. It also explains A2's apparent paradox that the lagged QoQ change was both negatively correlated with the target and the most important GB feature. Year-on-year change (12.0%), the eight-quarter mean (6.4%) and the price level (6.0%; dearer suburbs grew slightly less) come next. Market-wide signals (ABS growth lagged, regional momentum, NSW median) together contribute about 13.9%. The composition features we added for A4 (unit share and its change) contribute a modest 2.4%, and calendar seasonality almost nothing, consistent with A2's STL finding that seasonality explains only 0.4% of variance.

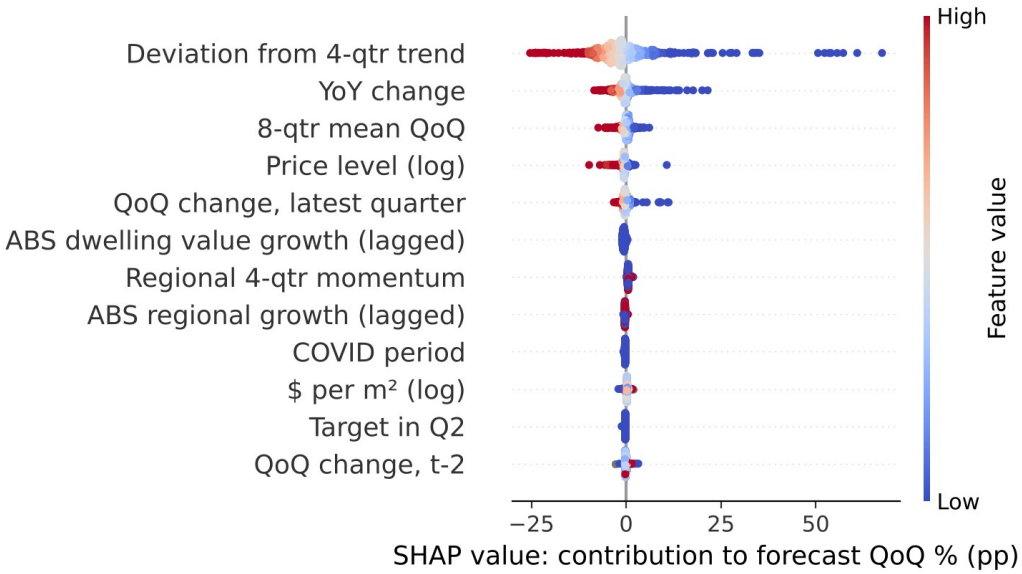


Figure A4.9 — *SHAP summary for LightGBM (h = 1, forecasts for 2026-Q2). Each dot is one suburb; colour is the feature value; position is its push on the forecast.*

3.7 Discussion of Results

Taken together, the evaluation supports four conclusions, and we think each is more useful than a single accuracy number.

- **Pooling across suburbs works, and it is the main source of skill.** The pooled models beat per-suburb SARIMA and Prophet decisively and significantly. That vindicates the A2 decision to move from per-suburb to pooled modelling.
- **Beyond one quarter, suburbs follow their region.** Suburb-specific predictability is short-lived and largely mean-reversion. For two- to four-quarter forecasts, the defensible output is the regional trend plus a wide, honest band, which is exactly what the app now shows.
- **The ceiling is measurement, not modelling.** Five well-tuned model families land within 0.1 pp of each other, and error tracks sampling noise almost perfectly. The next real gain will come from better price measurement, not a bigger model.
- **Calibrated uncertainty is achievable and cheap.** CQR delivers close to nominal coverage at every horizon and in every segment for the cost of two extra LightGBM fits.

Against the A1 problem statement, *can public sales data plus ABS data predict dwelling price changes one to four quarters ahead with acceptable accuracy?*, our answer is therefore qualified. **Yes for one quarter, and for direction at the regional level; only as a calibrated range for two to four quarters at suburb level.** We would rather state that clearly than present a four-quarter point forecast with false confidence.

3.8 Deployment

A2 ended with a working app that ran only on a team member's laptop. For A4 we treated deployment as an engineering problem with three separate concerns: what the app is allowed to see, how it is packaged, and how it stays current.

3.8.1 The Web Application (v4)

The app was rebuilt around the forecasting output rather than the back-test tables. Its main page (Figure A4.10) shows, for any of the 1,279 suburbs with enough recent sales, the observed median price history, a

four-quarter central forecast, the calibrated P10–P90 band as a shaded fan, and a reliability label (high/medium/low) based on recent sales volume and band width. Below the chart, a plain-language note states the typical back-tested error and explains *why* a suburb median can jump for reasons unrelated to any one home's value. We added it because, during our own review of the A2 app, we kept reading large forecast swings as market calls when they were usually a change in the mix of homes sold. Three further tabs cover the regional market, model accuracy and data lineage.

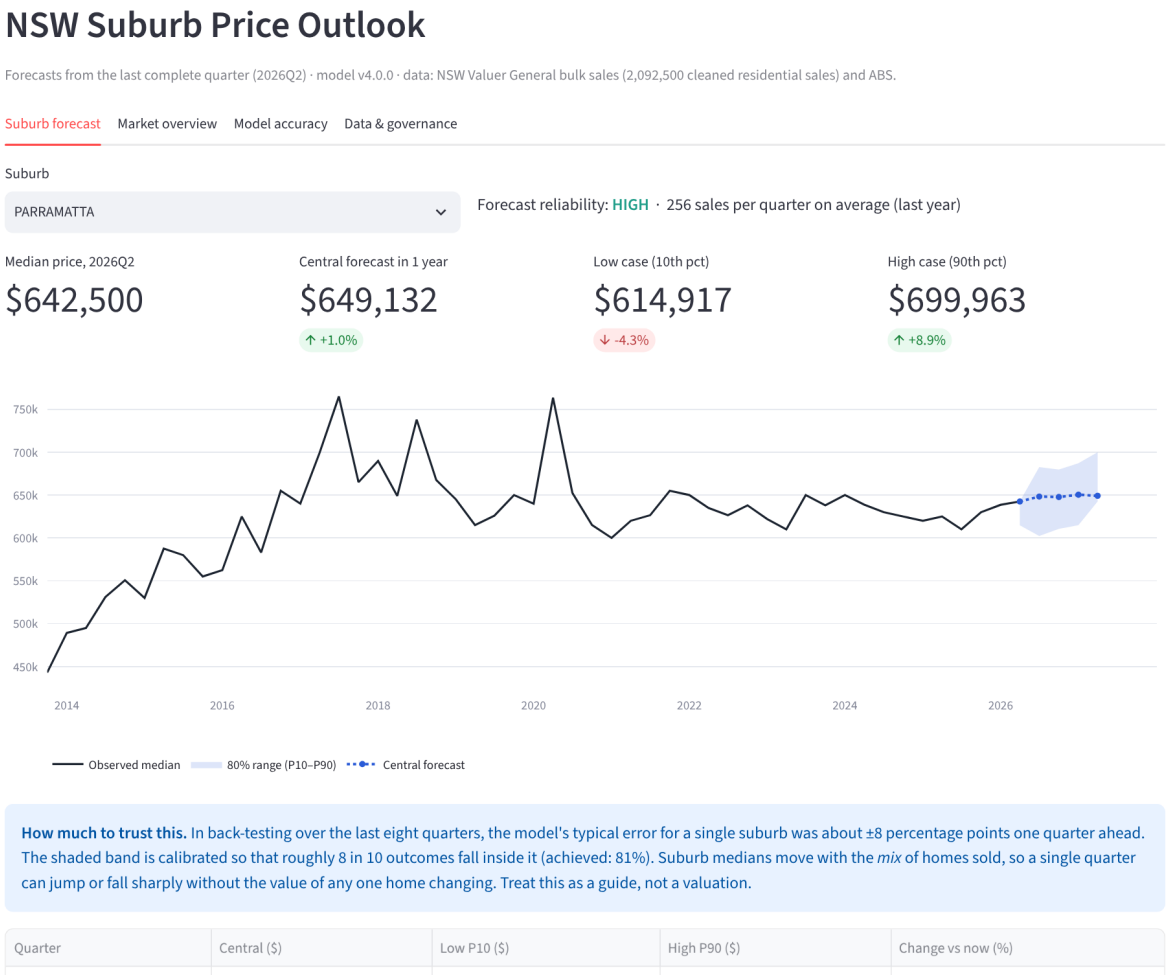


Figure A4.10 — *Web application v4, Suburb Forecast page (live screenshot, Parramatta). The shaded fan is the conformalised P10–P90 band from §3.5.4.*

A deliberate design decision is that **the app never connects to the database or to sale-level data**. A publishing step (`scripts/build_app_data.py`) exports five small, already-aggregated files to `app/data/` (0.57 MB in total), applying the five-sale suppression rule on the way out. This shrinks the attack surface to near zero, makes the app start in about two seconds, and means a deployed copy cannot leak an address even if misconfigured.



Figure A4.11 — *Model Accuracy tab: the same leaderboard and horizon comparison reported in §3.6, generated from the back-test tables.*

3.8.2 Packaging, Continuous Integration and Scheduling

Table A4.13 — *Deployment artefacts.*

Artefact	What it does	Status
`Dockerfile`	Python 3.11-slim image with only the five app dependencies; runs as a non-root user; health check on Streamlit's `/_stcore/health`	Written and reviewed
`requirements-app.txt`	Minimal runtime for Streamlit Community Cloud (the A2-recommended host)	Tested locally
`.github/workflows/ci.yml`	On every push/PR: install pinned deps, lint with ruff, run the test suite, apply both schema files to a throw-away PostgreSQL 16 service (twice, to prove idempotence)	Ready to activate on push
`.github/workflows/weekly_refresh.yml`	Every Tuesday 06:00 AEDT (after the Valuer General's Monday release): acquire → parse → clean → features → forecast → publish → load DB → test; commits refreshed app data, which triggers a Streamlit Cloud redeploy	Ready to activate on push
`Makefile` / pipeline CLI	`make all` reproduces every number in this report from raw archives	Run end-to-end for this report

The full pipeline run is not free. The statistical back-test dominates (about two hours on two CPU cores, almost all of it SARIMA and Prophet fitting). So the weekly job deliberately **excludes** the back-test: it re-trains and re-forecasts, which takes a few minutes, and the back-test is re-run at each tagged release. This keeps the published model and its quoted accuracy in step without paying for a full evaluation every week.

3.9 Deployment Readiness

We assessed readiness against a short checklist we would apply to any analytics product going in front of non-technical users (Table A4.14). Our honest summary: the system is **ready for a supervised public pilot** but not for unsupervised production use. The two gaps are organisational rather than technical: the

workflows have to be pushed to the team's GitHub to go live, and there is no named owner for the weekly job after the semester ends.

Table A4.14 — *Deployment readiness assessment.*

Criterion	Evidence	Ready?
Reproducible from raw data	Pipeline reproduces A2 Table 3.1 to within one record (§3.4.1)	Yes
Automated tests pass	13 tests, including look-ahead and suppression tests (Figure A4.12)	Yes
Accuracy known and published	Common-window back-test, shown in the app	Yes
Uncertainty calibrated	CQR bands hit 80.4–81.3% coverage against an 80% target	Yes
Privacy controls enforced in code	Suppression view, app reads aggregates only, role tests (Figure A4.3)	Yes
Model versioned and traceable	`model_registry` with SHA-256 of each artefact; CHANGELOG	Yes
Monitoring & alerting	Pipeline audit log exists; no automated drift alert yet	Partial
Live public URL	Config complete; requires push to team GitHub + Streamlit Cloud connect (~2 min)	Pending team action
Operational owner after semester	Not assigned	No

```
$ pytest -v
test_parser_reads_nested_archives PASSED
test_cleaning_rules_and_reconciliation PASSED
test_outliers_flagged_not_deleted PASSED
test_grid_is_complete_and_qoq_never_spans_a_gap PASSED
test_partial_quarter_has_no_target PASSED
test_features_do_not_look_ahead PASSED
test_abs_regressor_is_lagged_one_quarter PASSED
test_metrics_known_values PASSED
test_interval_scores_coverage PASSED
test_diebold_mariano_detects_better_model PASSED
test_sample_weight_is_capped_sqrt PASSED
test_full_run_reproduces_a2_table_3_1 PASSED
test_published_forecasts_are_ordered_and_suppressed PASSED
============================== 13 passed in 1.79s ==============================
```

Figure A4.12 — *Automated test suite (actual pytest output). The look-ahead test perturbs all post-origin sales and asserts the features at the origin are unchanged.*

3.10 Governance

Governance in A2 was mostly process: Jira epics, pull requests, a CHANGELOG. In A4 we pushed it into the artefacts themselves, so that the system enforces the rules rather than relying on people to remember them.

- **Model registry.** Every trained model is a row in `model_registry` with its version, horizon, the date it was trained through, its hyper-parameters, its back-test metrics and the SHA-256 hash of the saved file. Whatever produced a published forecast can be identified exactly.
- **Selection discipline.** Hyper-parameters were chosen on a validation window that closes before the test window opens (§3.5.2), and the test window was evaluated once. The champion (LightGBM) was fixed from the validation results, and the rule is recorded in `docs/MODEL_CARD.md`, so the test leaderboard could not quietly steer model choice.
- **Change control.** `docs/CHANGELOG.md` records every data-rule, schema, feature and model change with its effect on reported numbers, including all six audit findings. We recommend branch protection on `main` (one required review, CI must pass) as a repository setting; the steps are in `docs/DEPLOYMENT.md`.
- **Documentation as deliverable.** A model card (intended use, out-of-scope uses, metrics by segment, caveats; after Mitchell et al., 2019), a data-sources log with access dates and licences, and a security and privacy control matrix all ship in `docs/`.

Team process. Sprints 9 and 10 kept the A2 cadence: two-week sprints mirrored as Jira epics, a weekly stand-up on Teams, and a pull request per task with one reviewer. Table A4.15 shows final sprint status.

The ten-sprint plan from A1 is complete except the optional second-state extension, which we deliberately descoped in Sprint 9 to protect the audit and evaluation work.

Table A4.15 — *Final sprint status.*

Sprint	Focus	Status
1–8	Business understanding → data → EDA → models → back-test (A1/A2)	Complete
9	Audit of A2 artefacts; reproducible package; DB v2 and roles; feature store v2; tuning	Complete
10	Common-window back-test; uncertainty calibration; app v4; CI/CD; final report	Complete
Stretch	Second state (VIC) for generalisability	Descoped (see §3.13)

3.11 Ethics, Privacy and Security

The data is public under CC BY 4.0, but "public" is not the same as "harmless". Valuer General records include street addresses and exact prices. Combined with a suburb and a quarter, a thin market can make an "aggregate" effectively disclose one household's sale. Our controls are layered, so that no single mistake exposes anything:

- **Minimum cell size.** No statistic built from fewer than five sales is ever published. This is enforced in the database view, in the app-data export and by a test. It suppresses 55,883 suburb-quarters.
- **Least privilege.** Only the ETL role can read addresses; the app role can read forecasts and the privacy view only (Figure A4.3). The app image contains no credentials and no sale-level data.
- **No false precision.** Every forecast carries a calibrated band and a reliability label, low-liquidity suburbs are labelled *low* rather than hidden, and the app states plainly that it is not a valuation or financial advice.
- **Fairness across places.** We checked whether the model serves regional NSW worse than Sydney. Overall it does not: one-quarter-ahead MAE is 10.6 pp for regional suburbs and 10.9 pp for Sydney, and band coverage is 79.3% and 81.5%. Among liquid suburbs, regional forecasts are actually *more* accurate (6.6 vs 9.5 pp), plausibly because Sydney medians mix houses and apartments far more. Both regions show a small negative bias (forecasts 1.5–1.7 pp below outcomes on average) because the test window was a period of strong growth. We publish these segment results in the model card rather than averaging them away.
- **Secrets and supply chain.** Credentials come from environment variables and GitHub Secrets, `.env` is git-ignored, and all dependencies are pinned to the versions used to produce this report.

One ethical risk we cannot fully engineer away is **misuse as a buy/sell signal**. A forecast that suburb X will rise could, at scale, become self-fulfilling or be used in marketing. We mitigate this with framing (ranges first, central value second; "guide, not valuation"), and the model card lists investment advice and automated lending decisions as out of scope.

3.12 Limitations

- **The noise floor is real.** A suburb median from 20 sales has substantial sampling error, so a large part of each QoQ move is unforecastable composition noise (§3.7). Our forecasts are best read as *direction and range*, not a point estimate.
- **One state.** NSW only. The pipeline is config-driven, but other states publish in different formats and under different terms, so generalisability is untested.
- **Region proxy.** Sydney vs rest-of-NSW is still assigned by postcode range, not the ABS GCCSA correspondence, and suburbs are matched by name, so boundary changes over 2014–2026 are not tracked.
- **No macro drivers.** Interest rates, credit growth and population flows are absent: sourcing, aligning and lagging these series correctly was out of scope for the project window. The model will therefore react to a rate shock only after it shows up in sales.
- **Test window.** Eight quarters (2024-Q3 to 2026-Q2) is a reasonable but single regime. The model has not been tested through a sharp downturn.

3.13 Future Improvements

- **Repeat-sales or hedonic index per suburb.** Tracking the same properties over time (property_id is in the data) would strip out composition noise and could cut the noise floor substantially. This is the single most promising next step.
- **Hierarchical forecasting.** Reconciling suburb, region and state forecasts (e.g. MinT; Hyndman & Athanasopoulos, 2021) so they add up coherently and borrow strength across levels.
- **Macro features and scenario mode.** Add the RBA cash rate and ABS lending indicators, and let users ask "what if rates rise 50 bp?" with the band widening accordingly.
- **Drift monitoring.** Compare each new quarter's realised errors and band coverage with the back-test and alert when they leave the expected range.
- **Second state and SA2 geography.** Victoria's Valuer-General data, and the ABS postcode-to-SA2 correspondence to replace the postcode-range region proxy.

References (Assessment 4)

Australian Bureau of Statistics. (2026). *Total value of dwellings, Australia* (Cat. No. 6432.0). https://www.abs.gov.au/statistics/economy/price-indexes-and-inflation/total-value-dwellings/latest-release

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). https://doi.org/10.1145/2939672.2939785

Diebold, F. X., & Mariano, R. S. (1995). Comparing predictive accuracy. *Journal of Business & Economic Statistics, 13*(3), 253–263. https://doi.org/10.1080/07350015.1995.10524599

Harvey, D., Leybourne, S., & Newbold, P. (1997). Testing the equality of prediction mean squared errors. *International Journal of Forecasting, 13*(2), 281–291. https://doi.org/10.1016/S0169-2070(96)00719-4

Huber, P. J. (1964). Robust estimation of a location parameter. *The Annals of Mathematical Statistics, 35*(1), 73–101. https://doi.org/10.1214/aoms/1177703732

Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and practice* (3rd ed.). OTexts. https://otexts.com/fpp3/

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems, 30*, 3146–3154.

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems, 30*, 4765–4774.

Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I. D., & Gebru, T. (2019). Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency* (pp. 220–229). https://doi.org/10.1145/3287560.3287596

New South Wales Government. (2026). *Property sales information* [Data set]. NSW Valuer General. https://valuation.property.nsw.gov.au/embed/propertySalesInformation

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Romano, Y., Patterson, E., & Candès, E. (2019). Conformalized quantile regression. *Advances in Neural Information Processing Systems, 32*, 3543–3553.

Seabold, S., & Perktold, J. (2010). Statsmodels: Econometric and statistical modeling with Python. In *Proceedings of the 9th Python in Science Conference* (pp. 92–96). https://doi.org/10.25080/Majora-92bf1922-011

Taylor, S. J., & Letham, B. (2018). Forecasting at scale. *The American Statistician, 72*(1), 37–45. https://doi.org/10.1080/00031305.2017.1380080

Section 4 – Individual Work and Contribution Summary

4.1 Ahsan Uddin

Student ID: S386288

Individual Capstone Project Completion Evidence

[INSERT SCREENSHOT: Individual Capstone Project name, completion status, score, and completion date & time]

Individual Project Contribution (since Assessment 2)

I continued as project lead and owned acquisition and automation. I adapted the acquisition script into the package, wrote the weekly scheduled-refresh workflow that runs the pipeline every Tuesday after the Valuer General's release, restructured the repository around `src/`, `tests/` and `docs/`, wrote the CI workflow, and replaced the broken `requirements.txt`. I also ran Sprint 9–10 planning in Jira and the weekly stand-ups.

- Acquisition module · Weekly refresh workflow · CI workflow
- Jira Sprint 9–10 epics · Commit history

Professional Reflection

The most significant work this phase was turning a manual process into a scheduled one. The hard part was not the cron line but making the job safe to run unattended: it had to skip the two-hour back-test, fail loudly, and only publish if the tests passed. Coordinating four people through an audit that found defects in our own earlier work also needed care. We agreed early to treat findings as process issues rather than anyone's fault, and that kept the team moving. My contribution made sure the system keeps working after we stop touching it, which is what moves the project from coursework to something deployable.

4.2 Ferdous Anwar Anik

Student ID: S388272

Individual Capstone Project Completion Evidence

[INSERT SCREENSHOT: Individual Capstone Project name, completion status, score, and completion date & time]

Individual Project Contribution (since Assessment 2)

I led the Sprint 9 audit and the data-engineering rebuild. I wrote the nested-archive parser that reproduces A2's 2,762,903 raw records, corrected the cleaning module so it reproduces A2 Table 3.1, and vectorised outlier flagging (179 s → 5 s). I designed schema v2 (model registry, forecasts, audit log, privacy view), wrote the transactional COPY loader that repopulated `raw_sales` and `suburb_dim`, and implemented the least-privilege roles. I rebuilt the feature store on a complete grid with the lagged ABS regressor, and wrote the look-ahead and reconciliation tests.

- parsing.py · cleaning.py · features.py · db.py
- schema_v2.sql · tests · Jira tasks

Professional Reflection

The most significant thing I did was find that 9.5% of our own QoQ targets spanned a missing quarter. It was an uncomfortable discovery, because the A2 feature store was my work. The fix itself was simple; deciding to rebuild rather than patch, and then proving the new store had no look-ahead, took longer. Writing a test that deliberately corrupts future data taught me more about leakage than any lecture had. Because modelling and the app both sit on this layer, getting it right was what allowed the rest of the team to trust the numbers in this report. It also gave us a reproducible baseline, so any later change can be checked against known counts.

4.3 Mohd Yah Ya Raiyan

Student ID: S390934

Individual Capstone Project Completion Evidence

[INSERT SCREENSHOT: Individual Capstone Project name, completion status, score, and completion date & time]

Individual Project Contribution (since Assessment 2)

I led modelling and evaluation. I set up the validation design (separate tuning and test windows), ran the design and hyper-parameter search, and built the walk-forward back-test that scores 12 models on identical suburb-quarters across four horizons. That included re-running SARIMA and Prophet and reproducing the A2 Gradient Boosting model. I implemented conformalised quantile regression for the P10–P90 bands and the statistical evaluation: Diebold–Mariano tests, the liquidity analysis, segment errors and SHAP.

- backtest.py · models.py · evaluation.py
- Results tables · SHAP script · Jira tasks

Professional Reflection

The most important result I produced is also the most humbling: once models were compared fairly, the gap between them was much smaller than A2 suggested, and most of the remaining error is sampling noise in suburb medians. My main challenge was resisting the urge to keep tuning on the test set. Fixing the validation window first and evaluating the test window once made the final numbers believable, even when they were less impressive than A2's. A second challenge was compute: the SARIMA re-run alone took over two hours, so I had to plan runs carefully. Calibrating the uncertainty bands was the contribution I am proudest of, because it turns an honest-but-noisy forecast into something a user can actually act on.

4.4 Abrar Bin Khaiyum

Student ID: S395453

Individual Capstone Project Completion Evidence

[INSERT SCREENSHOT: Individual Capstone Project name, completion status, score, and completion date & time]

Individual Project Contribution (since Assessment 2)

I rebuilt the web application around forecasts with calibrated bands and reliability labels, and designed the publishing step that exports only aggregated, suppressed data to the app. I wrote the Dockerfile (non-root, health-checked) and the Streamlit Cloud configuration, and produced the model card, deployment guide and README. I consolidated this final report.

- app.py · build_app_data.py · Dockerfile
- Model card · Deployment guide · Jira tasks

Professional Reflection

The most significant work was redesigning the app so that uncertainty comes first: the shaded band, the reliability label and a plain-English accuracy note now sit beside every number. The technical challenge was keeping the app small and safe. Moving it from database access to a set of published files removed both the credential problem and the risk of leaking sale-level data. Writing the report taught me how much easier communication becomes when the evidence is reproducible: every figure here can be regenerated, so I could write with confidence. Working on deployment also showed me that most of the effort in a data product goes into the parts users never see: packaging, privacy rules and clear explanations.